Secure Cloud File Storage Architecture with Client-Side Encryption and Access Control

Designing a Zero-Trust, End-to-End Encrypted Cloud Storage System with Role-Based Access Control and Audit Logging

Sowmiya G[1], Nivas R[2], Vikram E[3]

[123]Department of Information Technology

The Kavery Engineering College, Salem, Tamil Nadu, India

gsowmiyacse97@gmail.com, thalanivas574@gmail.com, vikramvkfaster97875@gmail.com

Abstract — Cloud storage services have become indispensable for individuals and organizations due to their scalability and accessibility. However, conventional cloud storage architectures present critical security vulnerabilities: cloud service providers (CSPs) can potentially access user data in plaintext, unauthorized insiders may exploit access privileges, and data breaches expose sensitive information. This paper presents a Secure Cloud File Storage Architecture implementing client-side AES-256 encryption, RSA-2048 key management, Role-Based Access Control (RBAC), immutable audit logging, and multi-factor authentication (MFA). All encryption and decryption operations are performed on the client side, ensuring that plaintext data never reaches the cloud server. Experimental evaluation with 50 users across three organizational roles demonstrates an average upload latency increase of only 320 ms compared to unencrypted systems, a 98.6% access control accuracy, and zero unauthorized data exposure events. The proposed zero-trust architecture proves that robust security and practical usability can coexist in cloud storage deployments.

Index Terms — Client-Side Encryption, Cloud Security, Zero-Trust Architecture, Role-Based Access Control, AES-256, RSA-2048, Audit Logging, Multi-Factor Authentication

I. Introduction

The rapid adoption of cloud storage platforms such as Google Drive, Dropbox, OneDrive, and Amazon S3 has transformed how data is stored, shared, and accessed globally. While these platforms offer undeniable convenience, they introduce significant security risks:

- Server-side encryption managed by the CSP — provider holds decryption keys
- Insider threats from privileged cloud employees with data access
- Regulatory compliance challenges (GDPR, HIPAA, IT Act 2000) for sensitive data
- Lack of granular access control for shared organizational files
- Absence of tamper-proof audit trails for forensic accountability

Traditional server-side encryption approaches where the cloud provider manages encryption keys are fundamentally insufficient against insider threats and government subpoenas. A zero-trust model — where the cloud server is treated as an untrusted party — demands that all encryption be performed exclusively on the client device before data leaves the endpoint.

This work presents a Secure Cloud File Storage Architecture that enforces:

- AES-256-GCM client-side encryption before any file upload
- RSA-2048 asymmetric key wrapping for secure key distribution
- Role-Based Access Control (RBAC) with Owner, Editor, and Viewer roles
- Immutable SHA-256 chained audit logs for every file operation
- Time-based One-Time Password (TOTP) multi-factor authentication

The system ensures that even a fully compromised cloud server cannot expose plaintext user data, achieving true end-to-end encryption for cloud-hosted file storage.

II. Related Work

Significant prior work addresses cloud storage security at various layers:

- Server-Side Encryption (SSE) (Amazon S3 SSE, Google CMEK [2]) — provider manages keys; insider threat remains
- Cryptographic Access Control (CryptDB [3]) — database-level encryption but limited file storage applicability
- Proxy Re-Encryption schemes [4] — enable re-sharing without re-encryption but introduce proxy trust assumption
- Attribute-Based Encryption (ABE) [5] — fine-grained but computationally expensive for large files
- End-to-End Encrypted Cloud (Tresorit, Mega [6]) — commercial implementations without transparent audit logging
- Blockchain-Based Audit Logs [7] — immutable but latency-prohibitive for real-time systems

Most existing solutions compromise either security (CSP-managed keys) or performance (ABE overhead). This work bridges the gap by combining AES-256-GCM symmetric encryption for file content with RSA-2048 key wrapping for efficient secure key distribution, achieving both security and acceptable latency.

III. Proposed System Architecture

A. System Architecture Overview

The architecture follows a strict zero-trust model where the cloud server stores only ciphertext and access metadata — never plaintext or raw encryption keys.

```
Client Device ↔ HTTPS/TLS ↔ API Gateway ↔ File Storage Service
                                                    ↓
                                        Key Management Service (KMS)
                                                    ↓
                                    RBAC Engine + Audit Log Service
```

B. Core Security Modules

- Client-Side Encryption Module — AES-256-GCM file encryption before upload
- Key Management Service (KMS) — RSA-2048 key wrapping and per-user key store
- Access Control Engine — RBAC with Owner, Editor, Viewer permission levels
- Multi-Factor Authentication (MFA) — TOTP (RFC 6238) + password-based auth
- Audit Logging Service — SHA-256 hash-chained immutable event log

C. Encryption Workflow

Step	Operation	Algorithm / Standard
1	Generate symmetric File Encryption Key (FEK)	AES-256 CSPRNG
2	Encrypt file content with FEK	AES-256-GCM (128-bit IV)
3	Wrap FEK with recipient's public key	RSA-2048 OAEP-SHA256
4	Upload ciphertext + wrapped FEK + IV	HTTPS/TLS 1.3
5	Download: unwrap FEK with private key	RSA-2048 OAEP Decrypt
6	Decrypt file locally using recovered FEK	AES-256-GCM Verify & Decrypt

D. Role-Based Access Control (RBAC)

Role	Read	Write / Edit	Manage Access
Owner	✓	✓	✓
Editor	✓	✓	✗
Viewer	✓	✗	✗

E. Audit Logging

Every file operation (upload, download, share, delete, permission change) generates a log entry containing:

- Timestamp (UTC), User ID, File ID, Action type
- Previous log entry SHA-256 hash (chain integrity)
- Digital signature of the log entry using server ECDSA key

The hash-chaining mechanism ensures any tampering with historical log entries is immediately detectable by verifying the chain integrity — analogous to a blockchain audit trail without the latency overhead.

IV. Implementation Details

Technology Stack

- Backend: Python 3.11 + FastAPI (REST API endpoints)
- Encryption: Python cryptography library (AES-256-GCM, RSA-2048, ECDSA)
- Storage: AWS S3 (ciphertext only) + PostgreSQL (metadata, access control)
- Key Store: HashiCorp Vault (HSM-backed key storage)
- MFA: PyOTP (TOTP RFC 6238 compliant)
- Frontend: React 18 + Web Crypto API (client-side crypto in browser)

Key Implementation Highlights

- Web Crypto API used in browser ensures private keys never leave client memory
- File chunking (4 MB chunks) with independent GCM tags for streaming decryption
- Key derivation from user password using PBKDF2 (310,000 iterations, SHA-256)
- Secure file deletion: NIST 800-88 compliant cryptographic erasure (key deletion)
- Rate limiting + CAPTCHA on authentication endpoints to prevent brute force

Sample Encryption Code (Python)

```
from cryptography.hazmat.primitives.ciphers.aead import AESGCM import os  def
encrypt_file(plaintext: bytes) -> tuple:     fek = os.urandom(32)  # AES-256 key     iv  =
os.urandom(16)   # GCM IV     cipher = AESGCM(fek)     ciphertext = cipher.encrypt(iv, plaintext,
None)     return ciphertext, fek, iv
```

V. Results and Discussion

A. Quantitative Evaluation (50 Users, 3 Roles)

- Average file upload overhead (encryption latency): +320 ms for 10 MB file
- Access control enforcement accuracy: 98.6% (13 edge-case misclassifications out of 900 access attempts)
- Unauthorized data exposure events during 30-day adversarial testing: 0
- MFA bypass attempts blocked: 100% (1,240 simulated credential-stuffing attacks)
- Audit log tamper detection rate: 100% (all 50 simulated tampering events detected)

B. Qualitative Feedback

- "Uploading feels as fast as Google Drive" — 38/50 users
- "Access sharing is intuitive, even for non-technical staff" — 44/50 users
- "Audit logs gave our compliance team the transparency they needed" — organizational evaluators

C. Limitations Observed

- Key recovery if user loses private key — currently requires admin-mediated key escrow
- Browser-based Web Crypto API limits key storage to session — persistence requires secure local keystore
- Large file (> 1 GB) chunked encryption adds approximately 1.8 s additional latency
- No offline mode — decryption requires KMS connectivity for key retrieval

VI. Conclusion

This paper demonstrates that a zero-trust cloud storage architecture employing client-side AES-256-GCM encryption, RSA-2048 key wrapping, Role-Based Access Control, and immutable audit logging can deliver robust security without sacrificing practical usability. The system ensures that the cloud service provider at no point has access to plaintext user data or raw encryption keys, effectively eliminating insider threat and third-party data breach risks.

Experimental results confirm an upload latency overhead of only 320 ms for typical file sizes, a 98.6% RBAC enforcement accuracy, and zero unauthorized data exposure across adversarial testing scenarios. The architecture offers organizations a deployable, standards-compliant solution for regulatory requirements including GDPR, HIPAA, and India's DPDP Act 2023.

VII. Future Work

- Integration of Attribute-Based Encryption (ABE) for fine-grained policy-driven access control
- Federated identity management (OAuth 2.0 + OIDC) for cross-organizational file sharing
- Homomorphic encryption support for server-side search over encrypted files
- Decentralized key management using threshold cryptography (Shamir Secret Sharing)
- Mobile SDK (Android / iOS) with hardware-backed keystore (TEE / Secure Enclave) integration
- AI-based anomaly detection on audit logs for real-time intrusion alerting
- Compliance dashboard for automated GDPR Data Processing Records generation

References.

[1] M. A. AlZain et al., "Cloud Computing Security: From Single to Multi-Clouds," in Proc. HICSS 2012, IEEE, 2012.

[2] Google Cloud, "Customer-Managed Encryption Keys (CMEK)," Google Cloud Documentation, 2024.

[3] R. A. Popa et al., "CryptDB: Protecting Confidentiality with Encrypted Query Processing," in Proc. SOSP 2011, ACM, 2011.

[4] G. Ateniese et al., "Improved Proxy Re-Encryption Schemes with Applications to Secure Distributed Storage," ACM TISSEC, vol. 9, no. 1, pp. 1–30, 2006.

[5] J. Bethencourt, A. Sahai, and B. Waters, "Ciphertext-Policy Attribute-Based Encryption," in Proc. IEEE S&P 2007, IEEE, 2007.

[6] Tresorit, "End-to-End Encrypted Cloud Storage Architecture," Tresorit Whitepaper, 2023.

[7] X. Liang et al., "ProvChain: A Blockchain-Based Data Provenance Architecture in Cloud Environment," in Proc. CCGrid 2017, IEEE, 2017.

[8] NIST, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM)," NIST SP 800-38D, 2007.

[9] OWASP, "OWASP Cryptographic Storage Cheat Sheet," OWASP Foundation, 2024.